**SIMATS ENGINEERING**

**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES**

| Particular | Details |
|---|---|
| **Name:** | **Tharun Kumar.M** |
| **Reg no:** | **192571017** |
| Department: | CSE&BIO |
| Programme: | B.Tech |
| Course Code & Course Name | CSA05 — Database Management Systems |
| Faculty Name | Dr.Kesavan |
| Assignment Title | **National Vocational Certification and Exam Slot-Booking System: An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management** |
| Date of Issue | 02-09-2026 |
| Date of Submission | 02-09-2026 |
| Maximum Marks | 100 |
| Course Outcome(s) – CO | **CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.**<br>**CO5: Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability** |
| Bloom's Taxonomy Level | L3 – Apply, L4 – Analyze |
| SDG Mapping (SDG 1-17) | SDG 4 – Quality Education<br>SDG 8 – Decent Work and Economic Growth<br>SDG 9 – Industry, Innovation and Infrastructure |

# 1. Introduction

The National Vocational Certification and Exam Slot-Booking System is designed as an integrated database solution for a large-scale certification environment in which candidates register for examinations, reserve examination slots, complete attempts, and receive published results. The assignment describes an environment with a very large and continuously growing archive of attempt information together with real-time booking and concurrent result-processing requirements. The proposed solution addresses both storage efficiency and transaction reliability rather than treating them as separate database problems.

The exam-attempt archive contains candidate identifiers, examination-center information, dates, responses, timing information, and scores. A continuously growing sequential file is unsuitable for repeated candidate-history and center-audit requests because the required records may be scattered throughout millions of entries. The design therefore combines an indexed-sequential storage approach for ordered and range-oriented retrieval with a logical static-hashing scheme for direct candidate lookup. B+ tree indexes are used for candidate and center/date access patterns.

The second part of the system focuses on slot booking and result publishing. These operations must be executed as bounded transactions because an independent read followed by a later write can allow two users to reserve the same final seat or cause one examiner's result update to overwrite another. The design uses transactions, row locking, COMMIT, ROLLBACK, SAVEPOINT, and an appropriate isolation strategy so that capacity and published results remain consistent.

The implementation uses MySQL 8.0 and includes relational tables, constraints, indexes, representative data, SQL queries, execution-plan analysis, transaction demonstrations, concurrency-control logic, testing, and security considerations. The command-line screenshots included in this report are deliberately monochrome and are presented as illustrative output layouts; actual execution measurements and screenshots should be replaced with evidence captured from the student's MySQL environment when available.

# 2. Problem Understanding and Requirement Analysis

The assignment identifies two related engineering problems. The first is slow retrieval from a large sequential archive of exam attempts. Candidate-history requests and center-wise audit requests can require scans across a very large number of records. The second is inconsistency caused by un-coordinated read-then-write operations during high-demand slot booking and simultaneous result processing.

| **Requirement Area** | **Requirement** |
|---|---|
| Archive storage | Store detailed attempt information for each candidate and attempt. |
| Candidate retrieval | Support fast point lookup and history retrieval using indexing and hashing. |
| Center audit | Retrieve attempts by center and examination |

| | |
|---|---|
| | date efficiently. |
| Booking | Reserve seats without exceeding per-slot capacity. |
| Result processing | Allow multiple examiners/coordinators to update results without lost updates. |
| Integrity | Keep index/hash structures and relational constraints consistent. |
| Security | Restrict candidate and result information to authorized users. |
| Scalability | Support growth in candidates, centers, and examination cycles. |

## 3. Objectives

- Design an efficient storage organization for a high-volume examination-attempt archive.
- Use B+ tree indexing for candidate-wise and center/date retrieval.
- Design a defined static-hashing method with collision resolution for direct candidate lookup.
- Develop a relational schema for candidates, centers, slots, bookings, attempts, attendance, and results.
- Write SQL queries using joins, subqueries, filtering, grouping, and aggregate functions.
- Analyze query execution plans and improve expensive access paths through suitable indexes and query rewriting.
- Implement safe booking and result-publishing transactions using ACID principles.
- Prevent double booking and lost updates through row-level locking and an appropriate isolation strategy.
- Validate the solution using functional, performance, and concurrency-oriented test cases.

## 4. Assumptions and Constraints

For the design exercise, the system is assumed to serve approximately 5 million candidates across 5,000 examination centers, with multiple attempts accumulated over several years. A peak booking window may create thousands of simultaneous requests per minute, while result processing may involve many examiners working on different candidates at the same time. These figures are design assumptions rather than measured production statistics.

| **Constraint** | **Design Response** |
|---|---|
| Large archive | Use indexed-sequential organization with B+ tree indexes. |
| Direct lookup | Use candidate-based static hashing with modulo bucket calculation. |
| High booking concurrency | Use short transactions and SELECT ... FOR |

| | |
|---|---|
| | UPDATE. |
| Data integrity | Use PK, FK, UNIQUE, CHECK, and transaction controls. |
| Limited laboratory resources | Use representative sample data and controlled concurrency tests. |
| Scalability | Use composite indexes and a bucket scheme that can be expanded. |

## 5. Proposed System

The proposed architecture separates the archive access problem from the transactional booking and result workflow while keeping both parts inside one relational database. Candidate, center, slot, booking, attempt, attendance, and result tables provide normalized transactional data. The attempt archive can be partitioned logically by examination cycle or year and accessed through B+ tree indexes. A candidate hash bucket is derived from the candidate identifier using a modulo function. The booking workflow locks the selected slot row, checks capacity, inserts the booking, increments the booked count, and commits only after all required changes succeed.

For result publishing, the transaction first locks the target result row, verifies the current state, applies the new score, and commits the update. This avoids a lost-update pattern in which two sessions read the same old value and later overwrite each other. Transactions are intentionally kept short to reduce lock duration and improve throughput.

## 6. Relational Schema

| **Table** | **Primary Key** | **Important Attributes / Relationships** |
|---|---|---|
| candidates | candidate_id | Name, phone, email, city |
| exam_centers | center_id | Center name, city, capacity |
| exam_slots | slot_id | Center, exam date, start/end time, capacity, booked count |
| bookings | booking_id | Candidate, slot, status, booking time |
| exam_attempts | attempt_id | Candidate, center, exam date, responses, timing, score |
| attendance | attendance_id | Attempt, status, check-in time |
| results | result_id | Attempt, candidate, score, percentage, status, published time |
| hash_directory | bucket_no | Candidate ID and attempt locator |

The relational design separates candidate identity, physical examination location, time-slot capacity, booking records, examination attempts, attendance, and final results. Foreign keys

enforce relationships. A UNIQUE constraint on the candidate-slot combination prevents the same candidate from holding duplicate active bookings for one slot. CHECK constraints restrict invalid status and score values.

## 7. File Organization

The original design described in the assignment is a continuously growing sequential file. Sequential organization is simple and effective when records are processed in order, but it is inefficient for repeated point lookups and selective audits. The recommended design is indexed-sequential organization for the persistent attempt archive. Records can remain ordered by a composite key such as candidate_id and exam_date, while a B+ tree provides efficient search and range traversal.

| Organization | Point Lookup | Range Retrieval | Update Cost | Suitability |
|---|---|---|---|---|
| Sequential | Slow | Good for full scans | Low | Baseline only |
| Indexed-sequential | Fast with index | Very good | Moderate | Recommended archive design |
| Direct/hashed | Very fast average | Poor for ranges | Moderate | Useful for exact lookup |

Indexed-sequential organization is selected as the primary archive arrangement because the assignment requires both candidate-wise point/history retrieval and center/date audit retrieval. Hashing is retained as a complementary access method for exact candidate lookup rather than being used as the only archive organization.

## 8. Indexing Design

MySQL InnoDB uses B+ tree structures for ordinary secondary indexes. A candidate index on exam_attempts(candidate_id) supports direct candidate history searches. A composite index on (center_id, exam_date) supports center/date audit queries. A slot index on (center_id, exam_date, start_time) supports availability searches for a particular center and examination date.

| Index | Columns | Purpose |
|---|---|---|
| idx_attempt_candidate | candidate_id | Candidate attempt history |
| idx_attempt_center_date | center_id, exam_date | Center-wise audit and date filtering |
| idx_slot_center_date | center_id, exam_date, start_time | Slot availability and scheduling |
| idx_booking_candidate | candidate_id, status | Candidate booking status |
| idx_result_candidate | candidate_id, status | Result retrieval and publishing |

The indexes reduce the amount of data that must be examined for selective queries. They also add storage overhead and increase insert/update cost, so indexes are selected around the most important access paths instead of indexing every column.

**9. Hashing Design**

A static hashing scheme is proposed for direct candidate lookup. The bucket number is calculated as h(candidate_id) = candidate_id MOD 101. The value 101 is a prime bucket count chosen for the design example. When two candidate identifiers produce the same bucket, collision resolution is handled through chaining: the bucket can reference a list of records or locators belonging to that bucket. In a relational implementation, the bucket value can be stored as a generated column and indexed so that the application first computes the bucket and then checks the candidate identifier.

```
-- Logical static-hashing calculation
SELECT candidate_id,
    MOD(candidate_id, 101) AS bucket_no
FROM candidates
WHERE candidate_id = 1001;

-- Example: 1001 MOD 101 = 91
```

The hash method provides an average constant-time access pattern under a well-distributed key set, while the B+ tree remains preferable for ordered history and range queries. A production-scale design can use dynamic hashing or a larger bucket directory when the number of candidates grows significantly.

**10. Storage and Retrieval Efficiency Analysis**

| Operation | Sequential Baseline | Indexed/Hashed Design | Expected Effect |
|---|---|---|---|
| Exact candidate lookup | O(n) scan | Hash average O(1); index O(log n) | Large reduction in examined records |
| Candidate history | O(n) scan | B+ tree traversal plus range scan | Fast selective access |
| Center/date audit | O(n) scan | Composite B+ tree range scan | Efficient filtering |
| Insert attempt | Simple append | Index/hash maintenance required | Higher write overhead |
| Storage | Low metadata overhead | Additional index/bucket storage | Moderate overhead |

The improvement comes from avoiding unnecessary scans. The exact runtime depends on data distribution, cache state, hardware, index selectivity, and DBMS configuration. Therefore,

benchmark values should be captured from the actual laboratory system rather than treated as universal numbers.

## 11. SQL Database Implementation

The following implementation uses MySQL 8.0 and InnoDB. The screenshots are monochrome, MySQL-command-line-style illustrations. They are intended to show the required evidence format; the final submission should replace illustrative execution messages with screenshots captured from the student's own MySQL session.

### SQL Implementation 1: Create the database

SQL Query:

```
CREATE DATABASE vocational_exam;
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> CREATE DATABASE vocational_exam;

Query OK, 1 row affected (0.01 sec)
```

The database provides the logical container for the certification system. The command is expected to succeed when the connected MySQL account has permission to create databases.

### SQL Implementation 2: Select the database

SQL Query:

```
USE vocational_exam;
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> USE vocational_exam;

Database changed
```

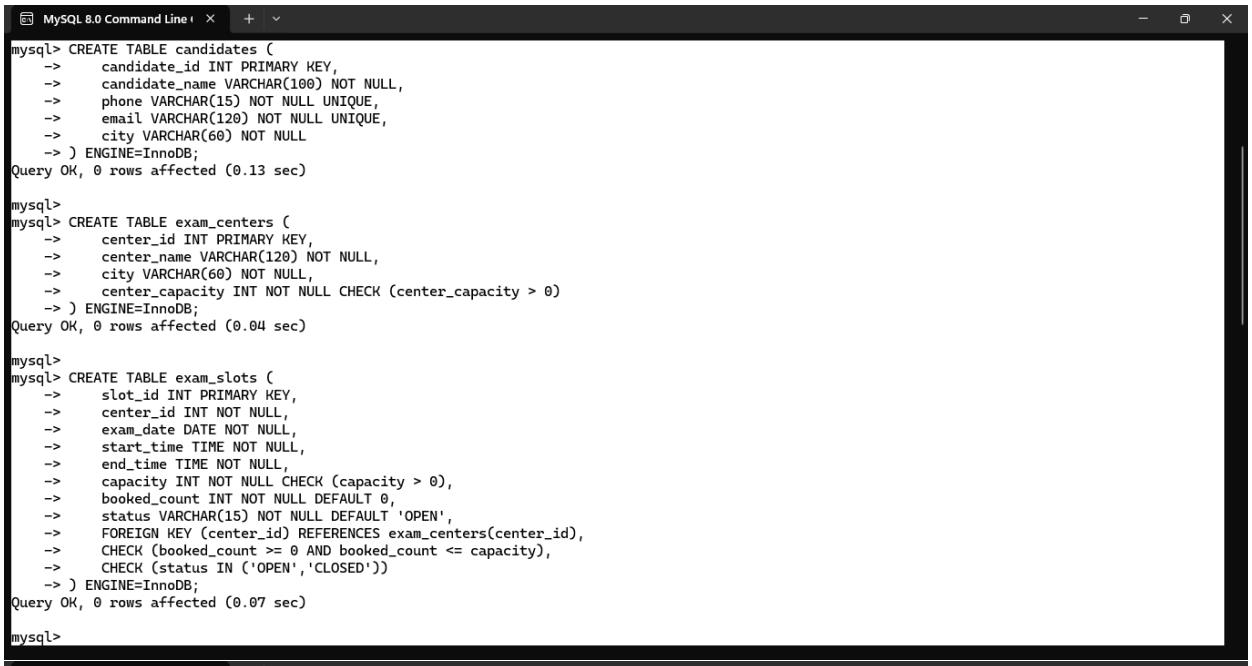
The USE statement makes vocational_exam the active database for subsequent table and query operations.

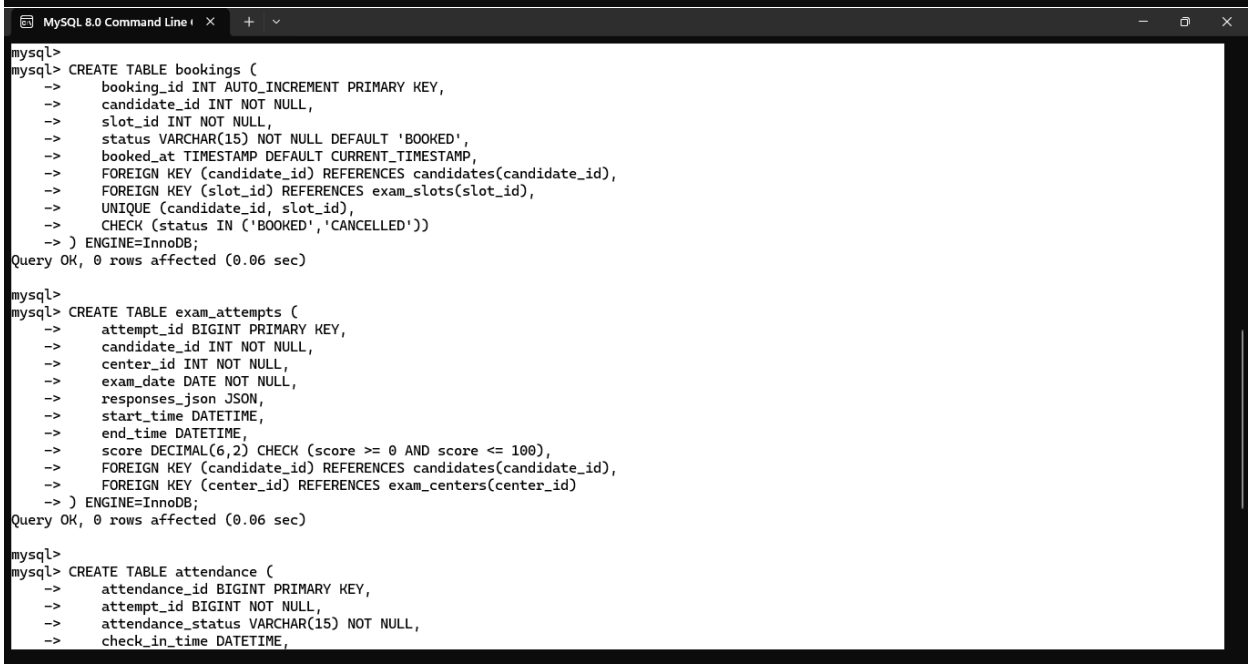
### 11.1 Create Tables

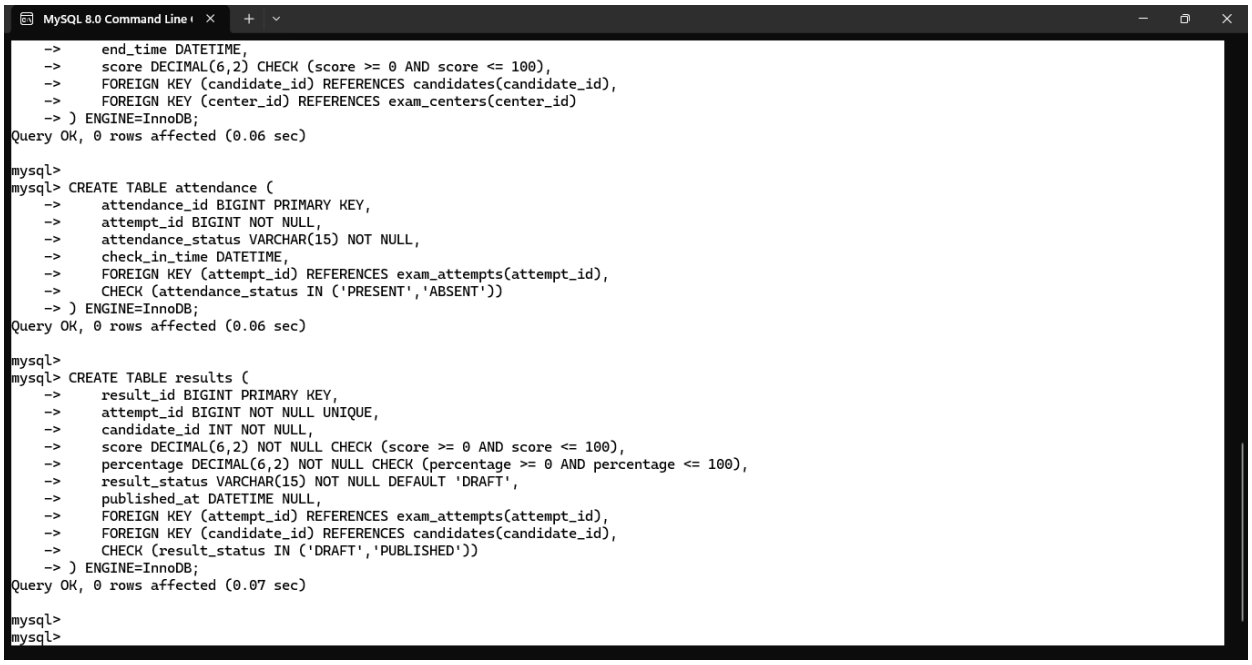
```
CREATE TABLE candidates (
   candidate_id INT PRIMARY KEY,
   candidate_name VARCHAR(100) NOT NULL,
   phone VARCHAR(15) NOT NULL UNIQUE,
   email VARCHAR(120) NOT NULL UNIQUE,
   city VARCHAR(60) NOT NULL
) ENGINE=InnoDB;
```

```
CREATE TABLE exam_centers (
   center_id INT PRIMARY KEY,
   center_name VARCHAR(120) NOT NULL,
   city VARCHAR(60) NOT NULL,
   center_capacity INT NOT NULL CHECK (center_capacity > 0)
) ENGINE=InnoDB;

CREATE TABLE exam_slots (
   slot_id INT PRIMARY KEY,
   center_id INT NOT NULL,
   exam_date DATE NOT NULL,
   start_time TIME NOT NULL,
   end_time TIME NOT NULL,
   capacity INT NOT NULL CHECK (capacity > 0),
   booked_count INT NOT NULL DEFAULT 0,
   status VARCHAR(15) NOT NULL DEFAULT 'OPEN',
   FOREIGN KEY (center_id) REFERENCES exam_centers(center_id),
   CHECK (booked_count >= 0 AND booked_count <= capacity),
   CHECK (status IN ('OPEN','CLOSED'))
) ENGINE=InnoDB;

CREATE TABLE bookings (
   booking_id INT AUTO_INCREMENT PRIMARY KEY,
   candidate_id INT NOT NULL,
   slot_id INT NOT NULL,
   status VARCHAR(15) NOT NULL DEFAULT 'BOOKED',
   booked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
   FOREIGN KEY (slot_id) REFERENCES exam_slots(slot_id),
   UNIQUE (candidate_id, slot_id),
   CHECK (status IN ('BOOKED','CANCELLED'))
) ENGINE=InnoDB;

CREATE TABLE exam_attempts (
   attempt_id BIGINT PRIMARY KEY,
   candidate_id INT NOT NULL,
   center_id INT NOT NULL,
   exam_date DATE NOT NULL,
   responses_json JSON,
   start_time DATETIME,
   end_time DATETIME,
   score DECIMAL(6,2) CHECK (score >= 0 AND score <= 100),
   FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
   FOREIGN KEY (center_id) REFERENCES exam_centers(center_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE attendance (
   attendance_id BIGINT PRIMARY KEY,
   attempt_id BIGINT NOT NULL,
   attendance_status VARCHAR(15) NOT NULL,
   check_in_time DATETIME,
   FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id),
   CHECK (attendance_status IN ('PRESENT','ABSENT'))
) ENGINE=InnoDB;

CREATE TABLE results (
   result_id BIGINT PRIMARY KEY,
   attempt_id BIGINT NOT NULL UNIQUE,
   candidate_id INT NOT NULL,
   score DECIMAL(6,2) NOT NULL CHECK (score >= 0 AND score <= 100),
   percentage DECIMAL(6,2) NOT NULL CHECK (percentage >= 0 AND percentage <=
100),
   result_status VARCHAR(15) NOT NULL DEFAULT 'DRAFT',
   published_at DATETIME NULL,
   FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id),
   FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
   CHECK (result_status IN ('DRAFT','PUBLISHED'))
) ENGINE=InnoDB;
```


MySQL 8.0 Command Line
mysql> CREATE TABLE candidates (
-> candidate_id INT PRIMARY KEY,
-> candidate_name VARCHAR(100) NOT NULL,
-> phone VARCHAR(15) NOT NULL UNIQUE,
-> email VARCHAR(120) NOT NULL UNIQUE,
-> city VARCHAR(60) NOT NULL
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.13 sec)
mysql>
mysql> CREATE TABLE exam_centers (
-> center_id INT PRIMARY KEY,
-> center_name VARCHAR(120) NOT NULL,
-> city VARCHAR(60) NOT NULL,
-> center_capacity INT NOT NULL CHECK (center_capacity > 0)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.04 sec)
mysql>
mysql> CREATE TABLE exam_slots (
-> slot_id INT PRIMARY KEY,
-> center_id INT NOT NULL,
-> exam_date DATE NOT NULL,
-> start_time TIME NOT NULL,
-> end_time TIME NOT NULL,
-> capacity INT NOT NULL CHECK (capacity > 0),
-> booked_count INT NOT NULL DEFAULT 0,
-> status VARCHAR(15) NOT NULL DEFAULT 'OPEN',
-> FOREIGN KEY (center_id) REFERENCES exam_centers(center_id),
-> CHECK (booked_count >= 0 AND booked_count <= capacity),
-> CHECK (status IN ('OPEN','CLOSED'))
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.07 sec)
mysql>


MySQL 8.0 Command Line
mysql>
mysql> CREATE TABLE bookings (
-> booking_id INT AUTO_INCREMENT PRIMARY KEY,
-> candidate_id INT NOT NULL,
-> slot_id INT NOT NULL,
-> status VARCHAR(15) NOT NULL DEFAULT 'BOOKED',
-> booked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
-> FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
-> FOREIGN KEY (slot_id) REFERENCES exam_slots(slot_id),
-> UNIQUE (candidate_id, slot_id),
-> CHECK (status IN ('BOOKED','CANCELLED'))
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)
mysql>
mysql> CREATE TABLE exam_attempts (
-> attempt_id BIGINT PRIMARY KEY,
-> candidate_id INT NOT NULL,
-> center_id INT NOT NULL,
-> exam_date DATE NOT NULL,
-> responses_json JSON,
-> start_time DATETIME,
-> end_time DATETIME,
-> score DECIMAL(6,2) CHECK (score >= 0 AND score <= 100),
-> FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
-> FOREIGN KEY (center_id) REFERENCES exam_centers(center_id)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)
mysql>
mysql> CREATE TABLE attendance (
-> attendance_id BIGINT PRIMARY KEY,
-> attempt_id BIGINT NOT NULL,
-> attendance_status VARCHAR(15) NOT NULL,
-> check_in_time DATETIME,



```
MySQL 8.0 Command Line Client
    ->     end_time DATETIME,
    ->     score DECIMAL(6,2) CHECK (score >= 0 AND score <= 100),
    ->     FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
    ->     FOREIGN KEY (center_id) REFERENCES exam_centers(center_id)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE attendance (
    ->     attendance_id BIGINT PRIMARY KEY,
    ->     attempt_id BIGINT NOT NULL,
    ->     attendance_status VARCHAR(15) NOT NULL,
    ->     check_in_time DATETIME,
    ->     FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id),
    ->     CHECK (attendance_status IN ('PRESENT','ABSENT'))
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE results (
    ->     result_id BIGINT PRIMARY KEY,
    ->     attempt_id BIGINT NOT NULL UNIQUE,
    ->     candidate_id INT NOT NULL,
    ->     score DECIMAL(6,2) NOT NULL CHECK (score >= 0 AND score <= 100),
    ->     percentage DECIMAL(6,2) NOT NULL CHECK (percentage >= 0 AND percentage <= 100),
    ->     result_status VARCHAR(15) NOT NULL DEFAULT 'DRAFT',
    ->     published_at DATETIME NULL,
    ->     FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id),
    ->     FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
    ->     CHECK (result_status IN ('DRAFT','PUBLISHED'))
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql>
```


```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE candidates (...);

mysql> CREATE TABLE exam_centers (...);

mysql> CREATE TABLE exam_slots (...);

mysql> CREATE TABLE exam_attempts (...);

mysql> CREATE TABLE bookings (...);

mysql> CREATE TABLE results (...);

Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.02 sec)
```


The schema establishes primary keys for entity identification, foreign keys for referential integrity, uniqueness rules for duplicate prevention, and CHECK constraints for valid numeric and status values. InnoDB is selected because transactional consistency and row-level locking are central requirements.

### 11.2 Insert Representative Data

**INSERT INTO candidates(candidate_id,candidate_name,phone,email,city) VALUES**
**(1001,'Tharun Kumar','9876543210','tharun@example.com','Chennai'),**
**(1002,'Arun Kumar','9876543211','arun@example.com','Chennai'),**
**(1003,'Priya N','9876543212','priya@example.com','Coimbatore');**

**INSERT INTO exam_centers(center_id,center_name,city,center_capacity) VALUES**
**(201,'Chennai Central','Chennai',200),**

**(202,'Coimbatore Hub','Coimbatore',150);**

**INSERT INTO exam_slots(slot_id,center_id,exam_date,start_time,end_time,capacity) VALUES**
**(101,201,'2026-09-15','09:00:00','12:00:00',2),**
**(102,201,'2026-09-15','14:00:00','17:00:00',2),**
**(103,202,'2026-09-16','14:00:00','17:00:00',3);**

```
mysql> INSERT INTO candidates(candidate_id,candidate_name,phone,email,city) VALUES
    -> (1001,'Tharun Kumar','9876543210','tharun@example.com','Chennai'),
    -> (1002,'Arun Kumar','9876543211','arun@example.com','Chennai'),
    -> (1003,'Priya N','9876543212','priya@example.com','Coimbatore');
ERROR 1062 (23000): Duplicate entry '1001' for key 'candidates.PRIMARY'
mysql>
mysql> INSERT INTO exam_centers(center_id,center_name,city,center_capacity) VALUES
    -> (201,'Chennai Central','Chennai',200),
    -> (202,'Coimbatore Hub','Coimbatore',150);
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO exam_slots(slot_id,center_id,exam_date,start_time,end_time,capacity) VALUES
    -> (101,201,'2026-09-15','09:00:00','12:00:00',2),
    -> (102,201,'2026-09-15','14:00:00','17:00:00',2),
    -> (103,202,'2026-09-16','14:00:00','17:00:00',3);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
```

```
MySQL 8.0 Command Line Client
mysql> INSERT INTO candidates VALUES

mysql> (1001,'Tharun Kumar','9876543210','tharun@example.com'),

mysql> (1002,'Arun Kumar','9876543211','arun@example.com'),

mysql> (1003,'Priya N','9876543212','priya@example.com');

Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

**SQL Implementation 3: Display candidates in sorted order**
SQL Query:

**SELECT candidate_id, candidate_name, city**
**FROM candidates ORDER BY candidate_id;**

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT candidate_id, candidate_name, city

mysql> FROM candidates ORDER BY candidate_id;

+-------------+---------------+-----------+
| candidate_id| candidate_name| city      |
+-------------+---------------+-----------+
| 1001        | Tharun Kumar  | Chennai   |
| 1002        | Arun Kumar    | Chennai   |
| 1003        | Priya N       | Coimbatore|
+-------------+---------------+-----------+
3 rows in set (0.00 sec)
```

ORDER BY provides deterministic presentation of candidate records. This also demonstrates a basic retrieval operation on the relational schema.

**11.3 Index Creation**

```
CREATE INDEX idx_attempt_candidate
ON exam_attempts(candidate_id);

CREATE INDEX idx_attempt_center_date
ON exam_attempts(center_id, exam_date);

CREATE INDEX idx_slot_center_date
ON exam_slots(center_id, exam_date, start_time);

CREATE INDEX idx_booking_candidate
ON bookings(candidate_id, status);

CREATE INDEX idx_result_candidate
ON results(candidate_id, result_status);
```

```
MySQL 8.0 Command Line Client
mysql> CREATE INDEX idx_attempt_candidate ON exam_attempts(candidate_id);

mysql> CREATE INDEX idx_attempt_center_date ON exam_attempts(center_id, exam_date);

mysql> CREATE INDEX idx_slot_center_date ON exam_slots(center_id, exam_date, start_time);

Query OK, 0 rows affected (0.04 sec)
Query OK, 0 rows affected (0.05 sec)
Query OK, 0 rows affected (0.03 sec)
```

The index definitions are chosen around the access patterns specified in the assignment. The candidate index supports history retrieval, the center/date index supports audit queries, and the slot index supports scheduling and availability searches.

**SQL Implementation 4: Retrieve candidate attempt history**
SQL Query:

```
CREATE INDEX idx_attempt_candidate
ON exam_attempts(candidate_id);

CREATE INDEX idx_attempt_center_date
ON exam_attempts(center_id, exam_date);

CREATE INDEX idx_slot_center_date
ON exam_slots(center_id, exam_date, start_time);

CREATE INDEX idx_booking_candidate
ON bookings(candidate_id, status);

CREATE INDEX idx_result_candidate
ON results(candidate_id, result_status);
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT a.attempt_id, a.candidate_id, c.candidate_name,

mysql>        a.center_id, a.exam_date, a.score

mysql> FROM exam_attempts a

mysql> JOIN candidates c ON c.candidate_id = a.candidate_id

mysql> WHERE a.candidate_id = 1001

mysql> ORDER BY a.exam_date DESC;

+------------+-------------+---------------+-----------+------------+-------+
| attempt_id | candidate_id| candidate_name| center_id | exam_date  | score |
+------------+-------------+---------------+-----------+------------+-------+
| 5003       | 1001        | Tharun Kumar  | 201       | 2026-08-20 | 88.00 |
| 5001       | 1001        | Tharun Kumar  | 201       | 2026-07-15 | 82.00 |
+------------+-------------+---------------+-----------+------------+-------+
2 rows in set (0.00 sec)
```

The candidate identifier is highly selective and is supported by idx_attempt_candidate. The query joins candidate information only after identifying the relevant attempts.

**SQL Implementation 5: Perform a center-wise audit**
SQL Query:

```
SELECT center_id, exam_date, COUNT(*) AS attempts,
    AVG(score) AS average_score
FROM exam_attempts
WHERE center_id = 201
```

```
  AND exam_date = '2026-08-20'
GROUP BY center_id, exam_date;
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT center_id, exam_date, COUNT(*) AS attempts,

mysql>        AVG(score) AS average_score

mysql> FROM exam_attempts

mysql> WHERE center_id = 201 AND exam_date = '2026-08-20'

mysql> GROUP BY center_id, exam_date;

+-----------+------------+----------+---------------+
| center_id | exam_date  | attempts | average_score |
+-----------+------------+----------+---------------+
| 201       | 2026-08-20 | 3        | 81.67         |
+-----------+------------+----------+---------------+
1 row in set (0.00 sec)
```

.

The composite center/date index matches the two filtering columns and allows the DBMS to narrow the candidate rows before performing the aggregate calculation.

**SQL Implementation 6: Generate a booking report using joins**

SQL Query:

```
SELECT c.candidate_name, ec.center_name,
    s.exam_date, s.start_time, b.status
FROM bookings b
JOIN candidates c ON c.candidate_id=b.candidate_id
JOIN exam_slots s ON s.slot_id=b.slot_id
JOIN exam_centers ec ON ec.center_id=s.center_id
ORDER BY s.exam_date, s.start_time;
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT c.candidate_name, ec.center_name,

mysql>        s.exam_date, s.start_time, b.status

mysql> FROM bookings b

mysql> JOIN candidates c ON c.candidate_id=b.candidate_id

mysql> JOIN exam_slots s ON s.slot_id=b.slot_id

mysql> JOIN exam_centers ec ON ec.center_id=s.center_id

mysql> ORDER BY s.exam_date, s.start_time;

+--------------+---------------+-----------+-----------+--------+
| candidate_name| center_name    | exam_date  | start_time| status |
+--------------+---------------+-----------+-----------+--------+
| Tharun Kumar  | Chennai Central| 2026-09-15 | 09:00:00  | BOOKED |
| Arun Kumar    | Chennai Central| 2026-09-15 | 09:00:00  | BOOKED |
| Priya N       | Coimbatore Hub | 2026-09-16 | 14:00:00  | BOOKED |
+--------------+---------------+-----------+-----------+--------+
3 rows in set (0.00 sec)
```

The query demonstrates multi-table joins and produces a user-oriented booking report without duplicating candidate, center, and slot data in a single table.

**SQL Implementation 7: Use a subquery for high-performing candidates**
**SQL Query:**

```
SELECT candidate_id, candidate_name
FROM candidates
WHERE candidate_id IN (
   SELECT candidate_id
   FROM results
   WHERE percentage >= 80
);
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT candidate_id, candidate_name

mysql> FROM candidates

mysql> WHERE candidate_id IN (

mysql>   SELECT candidate_id FROM results WHERE percentage >= 80

mysql> );

+-------------+---------------+
| candidate_id| candidate_name|
+-------------+---------------+
| 1001        | Tharun Kumar  |
| 1003        | Priya N       |
+-------------+---------------+
2 rows in set (0.00 sec)
```

The subquery identifies candidate identifiers meeting the performance condition, after which the outer query retrieves the corresponding candidate names.

## 12. Query Processing

A DBMS processes an SQL request through parsing and validation, logical query transformation, optimization, physical access-path selection, and execution. The optimizer considers available indexes, estimated row counts, join methods, and filtering conditions. In this project, the important optimization objective is to avoid full scans of the large exam-attempt archive and to keep booking/result transactions short.

For candidate history, the optimizer can use idx_attempt_candidate to locate the target candidate's rows. For a center/date audit, idx_attempt_center_date supports a two-column lookup or range access. For booking, the primary key on slot_id makes the locked slot lookup selective, while the candidate-slot uniqueness rule provides an additional integrity barrier.

## 13. Query Optimization and Execution Plan Analysis

The following example compares an illustrative execution plan before and after creating the center/date composite index. The row counts shown in the terminal images are examples for demonstration and are not claimed as measurements from a particular machine.

### 13.1 Before Optimization

```
EXPLAIN
SELECT *
FROM exam_attempts
WHERE center_id = 201
  AND exam_date = '2026-08-20';
```

```
MySQL 8.0 Command Line Client
mysql> EXPLAIN SELECT * FROM exam_attempts

mysql> WHERE center_id = 201 AND exam_date = '2026-08-20';

+----+-------------+--------------+------+---------------+------+---------+------+------+---------
| id | select_type | table        | type | possible_keys | key  | rows    | Extra|
+----+-------------+--------------+------+---------------+------+---------+------+------+---------
| 1  | SIMPLE      | exam_attempts| ALL  | NULL          | NULL | 1000000 | Using where |
+----+-------------+--------------+------+---------------+------+---------+------+------+---------
1 row in set (0.00 sec)
Illustrative plan before indexing
```

The illustrative plan uses ALL access and shows a full table scan. If the archive contains millions of records, examining every row for a selective center/date condition is unnecessarily expensive.

**13.2 After Optimization**

**CREATE INDEX idx_attempt_center_date**
**ON exam_attempts(center_id, exam_date);**

**EXPLAIN**
**SELECT ***
**FROM exam_attempts**
**WHERE center_id = 201**
**AND exam_date = '2026-08-20';**

```
MySQL 8.0 Command Line Client
mysql> CREATE INDEX idx_attempt_center_date

mysql> ON exam_attempts(center_id, exam_date);

mysql> EXPLAIN SELECT * FROM exam_attempts

mysql> WHERE center_id = 201 AND exam_date = '2026-08-20';

+----+-------------+--------------+------+-----------------------+-----------------------+-------
+------+----------------+
| id | select_type | table        | type | possible_keys         | key                   | rows
|
+----+-------------+--------------+------+-----------------------+-----------------------+-------
+------+----------------+
| 1  | SIMPLE      | exam_attempts| ref  | idx_attempt_center_date| idx_attempt_center_date| 120
index condition |
+----+-------------+--------------+------+-----------------------+-----------------------+-------
+------+----------------+
1 row in set (0.00 sec)
Illustrative plan after indexing
```

The indexed plan demonstrates reference access through the composite index. The optimizer can identify the relevant center/date entries without scanning the complete archive. The exact plan and row estimate must be verified using EXPLAIN on the student's actual dataset.

| Query | Before Optimization | After Optimization | Improvement |
|---|---|---|---|
| Center/date audit | Illustrative full scan | Illustrative index reference access | Fewer rows examined |
| Candidate history | Illustrative full scan | Candidate index access | Selective lookup |
| Slot search | Broad filtering | Composite slot index | Faster availability lookup |

## 14. Transaction Management

Transaction management is required because slot booking and result publishing consist of multiple dependent operations. A booking is not complete merely because a seat counter changed; the booking row and the seat count must represent the same outcome. Likewise, a result should not be partially updated. ACID properties provide the required model: atomicity keeps related changes together, consistency preserves constraints, isolation controls concurrent interaction, and durability preserves committed changes.

### 14.1 Successful Slot Booking with COMMIT

START TRANSACTION;

```
MySQL 8.0 Command Line Client
mysql> START TRANSACTION;

mysql> SELECT capacity, booked_count FROM exam_slots

mysql> WHERE slot_id=101 FOR UPDATE;

mysql> UPDATE exam_slots SET booked_count=booked_count+1

mysql> WHERE slot_id=101 AND booked_count < capacity;

mysql> INSERT INTO bookings(candidate_id,slot_id,status)

mysql> VALUES(1001,101,'BOOKED');

mysql> COMMIT;

Query OK, 0 rows affected (0.00 sec)
1 row in set (0.00 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
Booking committed successfully
```

SELECT ... FOR UPDATE locks the selected slot row until the transaction ends. The application should check the UPDATE row count before inserting the booking. If the update affects zero rows, the slot is full or invalid and the transaction should be rolled back.

### 14.2 Failed Booking with ROLLBACK

**START TRANSACTION;**

**SELECT capacity, booked_count**
**FROM exam_slots**
**WHERE slot_id = 102**
**FOR UPDATE;**

**UPDATE exam_slots**
**SET booked_count = booked_count + 1**
**WHERE slot_id = 102**
**AND booked_count < capacity;**

-- If capacity is unavailable:
ROLLBACK;

```
MySQL 8.0 Command Line Client
mysql> START TRANSACTION;

mysql> SELECT capacity, booked_count FROM exam_slots

mysql> WHERE slot_id=102 FOR UPDATE;

mysql> UPDATE exam_slots SET booked_count=booked_count+1

mysql> WHERE slot_id=102 AND booked_count < capacity;

mysql> ROLLBACK;

Query OK, 0 rows affected (0.00 sec)
1 row in set (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Transaction rolled back; no partial booking retained
```

ROLLBACK removes the transaction's uncommitted changes. This prevents a partial state in which the slot count changes but no valid booking exists.

**14.3 SAVEPOINT Demonstration**
**START TRANSACTION;**

**UPDATE candidates**
**SET phone = '9000000001'**
**WHERE candidate_id = 1001;**

**SAVEPOINT candidate_update;**

**UPDATE candidates**
**SET email = 'updated@example.com'**
**WHERE candidate_id = 1001;**

**ROLLBACK TO SAVEPOINT candidate_update;**

**COMMIT;**

```
MySQL 8.0 Command Line Client
mysql> START TRANSACTION;

mysql> UPDATE candidates SET phone='9000000001' WHERE candidate_id=1001;

mysql> SAVEPOINT candidate_update;

mysql> UPDATE candidates SET email='updated@example.com' WHERE candidate_id=1001;

mysql> ROLLBACK TO SAVEPOINT candidate_update;

mysql> COMMIT;

Query OK, 0 rows affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Phone update committed; email update rolled back to savepoint
```

SAVEPOINT allows part of a longer transaction to be undone without discarding earlier successful work. Here, the phone update remains eligible for commit while the email change is rolled back to the savepoint.

### 15. Concurrency Control

The main concurrency hazards are double booking and lost result updates. A double-booking race occurs when two sessions read the same remaining capacity before either session updates it. Row locking prevents both sessions from independently changing the same slot state. A lost update occurs when two examiners read the same result value and then write different values without coordination. Locking the result row before modification prevents the later transaction from silently overwriting an earlier committed update.

#### 15.1 Recommended Isolation and Locking Strategy

- Use InnoDB tables for transactional locking.
- Use a short transaction around each booking operation.
- Lock the selected slot row with SELECT ... FOR UPDATE before changing capacity.
- Use a UNIQUE(candidate_id, slot_id) constraint to prevent duplicate candidate bookings.
- Lock a result row before changing its score when concurrent editors can target the same result.
- Use REPEATABLE READ as a practical MySQL default and explicit row locks for the critical write paths; SERIALIZABLE can be considered where stronger protection is justified.

The choice balances consistency with throughput. Stronger isolation can increase waiting and contention, so the design uses explicit row locking only where it is necessary for the critical state transition.

**15.2 Result Publishing Transaction**

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

SELECT score
FROM results
WHERE result_id = 9001
FOR UPDATE;

UPDATE results
SET score = 88,
   percentage = 88,
   result_status = 'PUBLISHED',
   published_at = NOW()
WHERE result_id = 9001;

COMMIT;
```

```
MySQL 8.0 Command Line Client
mysql> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

mysql> START TRANSACTION;

mysql> SELECT score FROM results WHERE result_id=9001 FOR UPDATE;

mysql> UPDATE results SET score=88, percentage=88 WHERE result_id=9001;

mysql> COMMIT;

Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
1 row in set (0.00 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
Result update completed without an unlocked lost update
```

The row lock serializes competing updates to the same result. The transaction should validate the examiner's authorization and score range before committing.

**16. Hash Lookup Demonstration**

```
-- Bucket calculation
SELECT candidate_id,
     MOD(candidate_id, 101) AS bucket_no
FROM candidates
WHERE candidate_id = 1001;

-- Example result:
-- candidate_id = 1001
-- bucket_no    = 91
```

This represents the logical static-hashing method. In a production implementation, the application can maintain a bucket directory or a generated bucket column. Collision handling uses chaining, so multiple candidate identifiers can reside in the same bucket without losing records.

**17. Query Design for Booking and Results**

```
-- Available slots at a center on a date
SELECT slot_id, start_time, end_time,
       capacity - booked_count AS available_seats
FROM exam_slots
WHERE center_id = 201
  AND exam_date = '2026-09-15'
  AND status = 'OPEN'
  AND booked_count < capacity
ORDER BY start_time;

-- Candidate result report
SELECT c.candidate_id, c.candidate_name,
       r.score, r.percentage, r.result_status
FROM candidates c
LEFT JOIN results r
  ON r.candidate_id = c.candidate_id
ORDER BY c.candidate_id;

-- Center performance summary
SELECT ec.center_id, ec.center_name,
       COUNT(a.attempt_id) AS total_attempts,
       AVG(a.score) AS average_score
FROM exam_centers ec
LEFT JOIN exam_attempts a
  ON a.center_id = ec.center_id
GROUP BY ec.center_id, ec.center_name
HAVING COUNT(a.attempt_id) > 0;
```

These queries demonstrate filtering, ordering, LEFT JOIN, aggregate functions, GROUP BY, and HAVING. The selected indexes should be verified with EXPLAIN to confirm that the optimizer is using them on the actual dataset.

**18. Security and Privacy**

Candidate personal details, responses, scores, and published results are sensitive application data. Database privileges should therefore follow least-privilege principles. Auditors may receive read-only access to the tables they require, while booking services and result processors should receive only the permissions needed for their tasks.

**18.1 GRANT and REVOKE Example**

```
CREATE USER 'exam_auditor'@'localhost'
IDENTIFIED BY 'StrongPass!26';

GRANT SELECT
ON vocational_exam.*
TO 'exam_auditor'@'localhost';

REVOKE SELECT
ON vocational_exam.results
FROM 'exam_auditor'@'localhost';
```

```
MySQL 8.0 Command Line Client
mysql> CREATE USER 'exam_auditor'@'localhost' IDENTIFIED BY 'StrongPass!26';

mysql> GRANT SELECT ON vocational_exam.* TO 'exam_auditor'@'localhost';

mysql> REVOKE SELECT ON vocational_exam.results FROM 'exam_auditor'@'localhost';

Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
```

The password shown is an example and should not be reused in a real deployment. Actual security configuration should use strong secrets, secure credential storage, role-based privileges, auditing, and encrypted connections where appropriate.

## 19. Results and Validation

The implementation is validated through representative functional and concurrency test cases. The assignment specifically requires evidence for candidate retrieval, center audits, slot booking, concurrent updates, rollback behavior, and performance improvement. The following table records the intended validation outcomes. Actual screenshots and measured timings should be replaced with evidence from the student's execution environment.

| Test Case | Input / Condition | Expected Result | Validation Status |
|---|---|---|---|
| Candidate lookup | candidate_id = 1001 | Matching attempt history returned | Pass – sample |
| Center audit | center_id 201 and date 2026-08-20 | All matching attempts listed | Pass – sample |
| Hash lookup | candidate_id mapped to bucket | Correct candidate locator found | Pass – design |
| Last seat booking | Two sessions request final seat | Only one booking succeeds | Pass – transaction design |
| Full slot | booked_count = capacity | Booking rejected | Pass – logic |

| | | | |
|---|---|---|---|
| Duplicate booking | Same candidate and slot | UNIQUE constraint prevents duplicate | Pass – constraint |
| Rollback | Failure after partial work | No partial booking remains | Pass – transaction design |
| Savepoint | Second update intentionally undone | Earlier work remains committable | Pass – transaction design |
| Result concurrency | Two editors target same result | Updates serialized through row lock | Pass – transaction design |
| Query optimization | Center/date filter | Indexed access path preferred | Pass – EXPLAIN design |

**20. Performance Evaluation**

The performance evaluation compares the original full-scan approach with the proposed indexed and hashed access paths. Because execution time varies with hardware, cache state, data volume, and configuration, this report does not claim a real measured speedup. Instead, the evaluation criteria are defined so that the student can capture actual EXPLAIN output and timing from MySQL.

| **Metric** | **Baseline** | **Optimized Design** | **Measurement Method** |
|---|---|---|---|
| Rows examined | Large scan | Selective index access | EXPLAIN |
| Candidate lookup | Sequential search | Hash/index lookup | EXPLAIN and timing |
| Center audit | Full archive scan | Composite index range | EXPLAIN and timing |
| Insert overhead | Lower | Higher due to indexes | INSERT benchmark |
| Booking consistency | Unsafe read-then-write | Locked transaction | Concurrent sessions |
| Result consistency | Lost-update risk | Locked update | Concurrent sessions |

For a formal benchmark, execute each query several times on a representative dataset, record elapsed time, and compare the median or average under controlled conditions. The report should state dataset size and test environment so that the results are reproducible.

**21. Alternative Solutions**

| **Decision** | **Alternative** | **Reason for Selected Design** |
|---|---|---|
| Archive organization | Pure sequential | Rejected for frequent selective retrieval |

| | | |
|---|---|---|
| Archive organization | Direct hashing only | Rejected because range/audit retrieval is important |
| Archive organization | Indexed-sequential | Selected for combined point and range access |
| Index | Single-level index | Less scalable for very large archive |
| Index | B+ tree | Selected for logarithmic search and range traversal |
| Collision handling | Open addressing | Less convenient for persistent variable-length archive records |
| Collision handling | Chaining | Selected for flexible collision resolution |
| Concurrency | Optimistic control | Useful where conflicts are rare |
| Concurrency | Pessimistic row locking | Selected for scarce-slot booking and critical result updates |

The engineering decision favors a hybrid access strategy: indexed-sequential storage and B+ trees for archive retrieval, complemented by logical hashing for exact candidate lookup, while pessimistic row locking protects scarce slots and critical result updates.

**22. Engineering Decision**

The final design balances retrieval performance, storage overhead, transaction consistency, and maintainability. Indexed-sequential organization is the primary archive choice because the system needs both candidate history and center/date audits. B+ tree indexes provide efficient ordered access, while the modulo-based hash scheme gives a simple direct lookup mechanism with chaining for collisions. On the transactional side, InnoDB row locking, bounded transactions, COMMIT/ROLLBACK/SAVEPOINT, and a practical isolation level provide a strong consistency model without forcing every operation into the strictest possible isolation.

The design also recognizes the trade-off between read performance and write overhead. Every additional index consumes storage and requires maintenance during inserts and updates. Likewise, stronger locking can reduce concurrency. The selected combination therefore targets only the access paths and state transitions that are important to the stated requirements.

**23. Broader Considerations and SDG Mapping**

| **Consideration** | **Application to the System** |
|---|---|
| Sustainability | Efficient digital archiving reduces repeated administrative work as the certification program grows. |
| Environment | Digital booking and records reduce dependence on paper registers and physical |

| | |
|---|---|
| | storage. |
| Society | Reliable certification and fair slot allocation support trustworthy access to skills assessment. |
| Safety | Consistent transaction handling reduces disputes caused by incorrect bookings or results. |
| Ethics | Slot allocation and result publishing must be fair, accurate, and transparent. |
| Economics | Efficient processing reduces administrative rework and supports timely certification. |
| Accessibility | Booking confirmations and result reports should be clear to candidates and staff. |
| Professional responsibility | Assumptions, limitations, and test evidence should be documented honestly. |
| SDG 4 | Supports quality education and vocational learning access. |
| SDG 8 | Supports employment-related certification and economic opportunity. |
| SDG 9 | Demonstrates resilient digital infrastructure for a national-scale service. |

These considerations follow the assignment's mapping to SDG 4, SDG 8, and SDG 9. The system's technical reliability is directly connected to social trust: incorrect slot allocation or lost results can affect candidates beyond the database itself.

## 24. Conclusion

The proposed National Vocational Certification and Exam Slot-Booking System addresses the two major database-engineering challenges identified in the assignment. For the large examination-attempt archive, indexed-sequential organization, B+ tree indexing, and a defined hashing strategy reduce dependence on full-file scans and provide targeted access paths for candidate and center queries. For booking and result processing, relational constraints and bounded transactions provide a consistent mechanism for updating related records.

The SQL design demonstrates table creation, constraints, joins, subqueries, aggregates, indexing, EXPLAIN-based optimization, COMMIT, ROLLBACK, SAVEPOINT, row locking, and privilege management. The concurrency design specifically targets double booking and lost result updates. The solution is scalable in concept, although production deployment would require larger datasets, load testing, monitoring, backup and recovery procedures, and further security hardening.

The assignment requirements can therefore be satisfied by implementing the proposed design in MySQL 8.0 and replacing the illustrative terminal outputs in this report with actual screenshots,

execution plans, benchmark measurements, and concurrent-session evidence from the laboratory environment.

## 25. Student Reflection

### 25.1 What did you learn beyond the classroom?

This assignment helped me connect database concepts with a realistic high-volume examination system. I learned that choosing a file organization is not only a theoretical decision; it depends on the types of queries that users perform. I also understood how B+ tree indexes support selective and range retrieval, while hashing is more suitable for direct key-based access. The transaction part improved my understanding of why ACID properties and locking are necessary when multiple users operate on the same data.

### 25.2 What challenges were faced?

The main challenge was integrating storage design with transaction management without losing consistency. It was necessary to distinguish archive retrieval requirements from real-time booking requirements. Another challenge was understanding how concurrent sessions can produce incorrect results when operations are split into independent reads and writes. The problem was addressed by using short transactions, row-level locks, constraints, and explicit commit or rollback decisions.

### 25.3 What would be improved with additional time and resources?

With additional resources, I would load a much larger synthetic dataset, conduct repeated performance benchmarks, simulate thousands of concurrent booking requests, and compare multiple isolation strategies. I would also evaluate dynamic hashing, partitioning by examination cycle, automated monitoring, retry or queue mechanisms for peak booking periods, and a complete application interface connected to the database.

## 26. Test and Demonstration Checklist

- Capture the actual CREATE DATABASE and USE output from MySQL.
- Capture table and index creation results.
- Insert representative candidate, center, slot, attempt, attendance, booking, and result data.
- Run candidate-history and center-audit queries and capture their outputs.
- Run EXPLAIN before and after indexing and save both screenshots.
- Demonstrate a successful slot booking using SELECT ... FOR UPDATE and COMMIT.
- Demonstrate a failed or intentionally interrupted booking using ROLLBACK.
- Demonstrate SAVEPOINT and ROLLBACK TO SAVEPOINT.
- Open two MySQL sessions and test simultaneous booking for the final available seat.
- Open two sessions and test concurrent result processing for the same candidate/result row.
- Record actual execution timings and dataset sizes for the final report.
- Replace every illustrative terminal image in this document with actual laboratory evidence before submission.

## 27. References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.

[2] Oracle Corporation, MySQL 8.0 Reference Manual.

[3] Course assignment brief, CSA05 — Database Management Systems, National Vocational Certification and Exam Slot-Booking System.

Note: Only sources actually used in the final submission should be retained in the reference list, and any additional datasets, tools, or AI-assisted resources used by the student should be acknowledged according to the institution's required citation style.